

4.22.2.1. Nonce

The nonce used to encrypt the encrypted fields of the payload SHALL be the first [CRYPTO_AEAD_NONCE_LENGTH_BYTES](#) bytes of the message payload. The nonce is included in plaintext.

4.22.2.2. Encrypted Section

The encrypted section SHALL be a concatenation of the following, in order:

1. The [Check-In Counter](#) value used in the encryption process, represented as a 4-byte little-endian value.
2. The Application Data, if used by the use case.

Check-In Counter

The [Check-In Counter](#) value SHALL be used to generate the nonce. This value SHALL be provided by the Check-In Protocol use case.

Application Data

The application data is defined and provided by the Check-In Protocol use case. The Check-In Protocol use case is not required to use application data. If application data is not used for a specific use-case, creation of a Check-In message SHALL use a zero-length byte array for the application data.

4.22.2.3. MIC

Message integrity check generated by the AEAD is appended after the encrypted section. It SHALL be the last [CRYPTO_AEAD_MIC_LENGTH_BYTES](#) bytes.

4.22.3. Algorithms

The Check-In message is a sessionless message, since one of the goals of the protocol is to provide a means to recover a secure session that was lost. Since no session keys are available to encrypt the entire message, the encryption of the Check-In message is limited to part of the payload.

The inputs provided to the encryption process by the Check-In protocol use case are:

1. The value of [Check-In Counter](#) represented as 4-byte little-endian value. In the following steps, we assume the counter value is in the correct format.
2. The application data bytes represented as a byte array. In the following steps, we assume the application data is in the correct format.
3. A symmetric encryption key represented as a [CRYPTO_SYMMETRIC_KEY_LENGTH_BYTES](#)-byte array. In the following steps, we assume the key is in the correct format.

4.22.3.1. Nonce Generation

To generate the nonce, the Check-In message SHALL use the [Keyed-Hash Message Authentication Code](#) algorithm. The inputs of the algorithm SHALL be the key and the [Check-In Counter](#) value. Both will be provided by the Check-In Protocol use case. From the generated hash, the nonce SHALL be the first [CRYPTO_AEAD_NONCE_LENGTH_BYTES](#) bytes to match the AEAD requirements.